Check for updates

Wiley
IET Software
Volume 2025, Article ID 9238663, 25 pages
https://doi.org/10.1049/sfw2/9238663

Research Article

Blockchain-Based Model to Predict Agile Software Estimation Using Machine Learning Techniques

Mohammad Ayub Latif,[1] **Muhammad Khalid Khan**,[1] **Maaz Bin Ahmad**,[1] **Toqeer Mahmood**,[2] **Muhammad Tariq Mahmood**,[3] **and Young-Bok Joo**[3]

[1]*College of Computing and Information Sciences, Karachi Institute of Economics and Technology, North Nazimabad Site, Karachi, Pakistan*
[2]*Department of Computer Science, National Textile University, Faisalabad 37610, Pakistan*
[3]*School of Computer Science and Engineering, Korea University of Technology and Education, 1600 Chungieolro, Byeongcheonmyeon, Cheonan 31253, Republic of Korea*

Correspondence should be addressed to Young-Bok Joo; ybjoo@koreatech.ac.kr

Received 9 February 2025; Revised 29 August 2025; Accepted 15 September 2025

Guest Editor: Nadeem Sarwar

Copyright © 2025 Mohammad Ayub Latif et al. IET Software published by John Wiley & Sons Ltd. This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

The importance of software estimation is utmost, as it is one of the most crucial activities for software project management. Although numerous software estimation techniques exist, the accuracy achieved by these techniques is questionable. This work studies the existing software estimation techniques for Agile software development (ASD), identifies the gap, and proposes a decentralized framework for estimation of ASD using machine-learning (ML) algorithms, which utilize the blockchain technology. The estimation model uses nearest neighbors with four ML techniques for ASD. Using an available ASD dataset, after the augmentation on the dataset, the proposed model emits results for the completion time prediction of software. Use of another popular dataset for ASD predicts the software effort using the same proposed model. The crux of the proposed model is that it simulates blockchain technology to predict the completion time and the effort of a software using ML algorithms. This type of estimation model, using ML, making use of blockchain technology, does not exist in the literature, and this is the core novelty of this proposed model. The final prediction of the software effort integrates another technique for improving the calculated estimation, the standard deviation technique proposed by the authors previously. This model helped lessening the overall mean magnitude of relative error (MMRE) of the original model from 6.82% to 1.73% for the augmented dataset of 126 projects. All four ML techniques used for the proposed model give a better p-value than the original model using statistical testing through the Wilcoxon test. The average of the MMRE for effort estimation of all four techniques is below 25% on a dataset of 136 projects. The application of the standard deviation technique further helps in lessening the MMRE of the proposed model at 70%, 80%, and 90% confidence levels. The work will give insight to researchers and experts and open the doors for new research in this area.

1. Introduction

Project size, software development methodology, the stage of calculating the software effort, and the accuracy we are targeting are the four factors that are the core when deciding about the effort estimation model for a particular software under development [1]. Figure 1 shows the four major factors for selecting the software estimation model. The stakeholders require calibration to understand the output of the estimation model into something meaningful. This calibration requires data. For example, the lines of code (LOC) count output cannot be converted into effort for persons-month, persons-week, or persons-days unless and until calibration is performed. McConnell [1] has evidence that calibrating with project data (which is the data of the same project that is undergoing development) instead of industrial or historical data (which constitutes the data of a different organization or a different software of the same organization, respectively), yields the most accurate results. The only limitation of using project data is that the development methodology should be iterative so that the

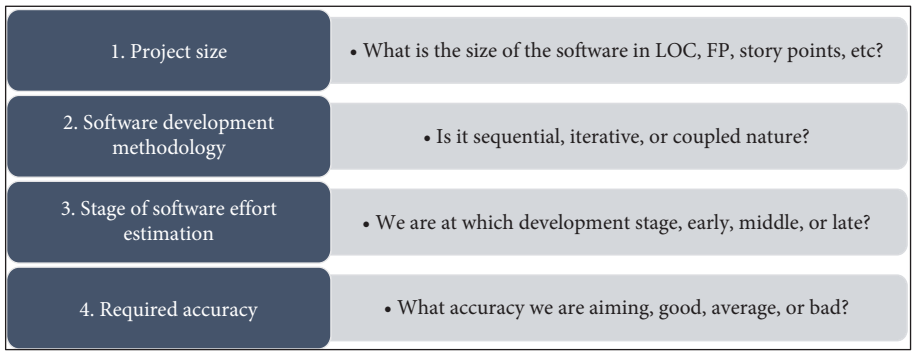

FIGURE 1: Factors that need to be considered while selecting an effort estimation model for software [1].

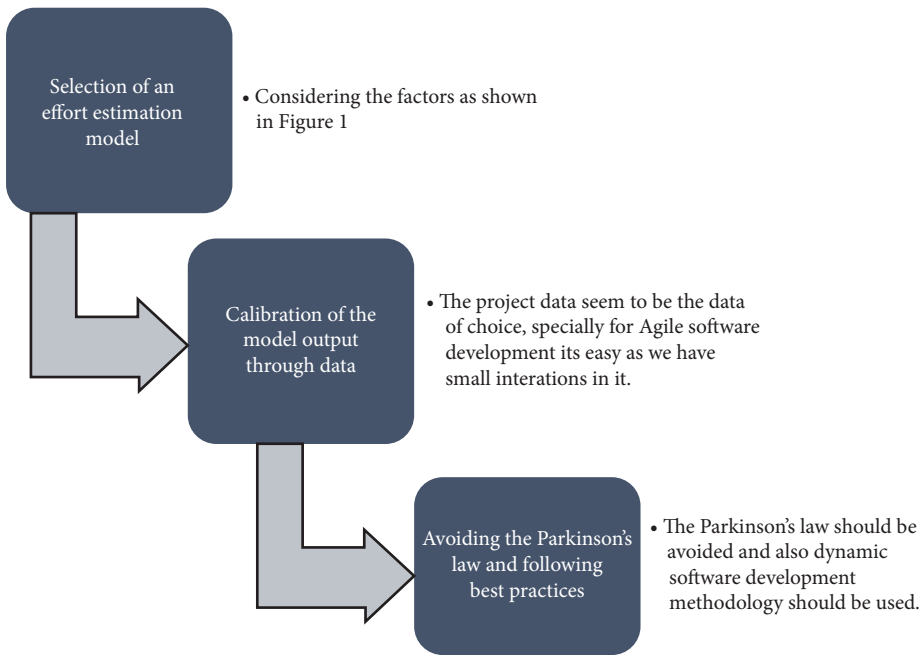

FIGURE 2: Three important cornerstones for performing software effort estimation for an ongoing project.

data from a previous iteration can be used for calibrating the iteration that comes after the first one. Researchers have shown the use of project data compared to industrial and historical data as a best practice for achieving accuracy in software estimation [2]. Avoiding Parkinson's law in an ongoing project where the work extends to the given deadline, even when the deadline is very relaxed, should be a concern for the project managers, as this can unnecessarily delay an ongoing project [3, 4]. Dynamic software estimation models where the same number of practitioners is not fixed for the whole project, but the number of practitioners can vary for phases, are mostly used in this modern era [1, 2, 5]. The two categories of software effort estimation are algorithmic and non-algorithm models. LOC, function points (FPs), and the constructive cost model (COCOMO) are common examples of algorithm models. At the same time, expert judgment, analogy-based techniques, proxy techniques, pricing to win, etc., would fall under the non-algorithm category [6]. The use of software effort estimation shows the immense use of machine-learning (ML) techniques. It will not be wrong to state them as the third category for software effort estimation.

Research has classified these ML-based models under computational intelligence [7, 8]. So, the three essential cornerstones for estimating an ongoing project's effort, depicted as a small flowchart in Figure 2. Blockchain is a kind of distributed database shared among users, and it is different from a database in terms of the mechanism it uses for storage. The simple idea is that the data are stored in blocks, and cryptography connects those blocks. They call it a chain, as the blocks are linked together to form a chain. The use of smart contracts for many different kinds of business areas has also grown over time. Smart contracts are stored on a blockchain network and executed immediately when a contract's terms and conditions are meet. Wide use of blockchain is evident in software project management. Each node or computer can host project data in a distributed ledger. Each node will update the data, and no one can alter it. It helps achieve compliance, immutability, and transparency in the recorded data. As there is

iet sfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

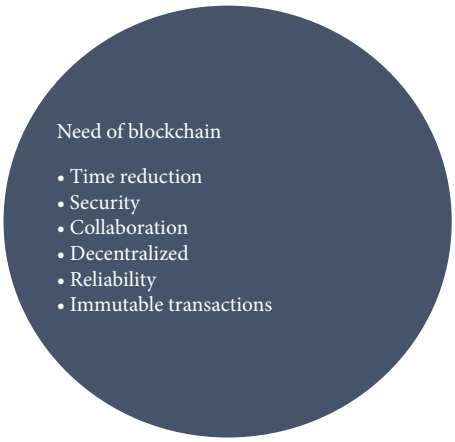

FIGURE 3: Advantages of a blockchain architecture.

transparency, the data are unaltered, and unaltered data ensures accuracy.

The AEC industry integrated blockchain technology to achieve better performance in projects and control the challenges related to project management, such as schedule delays, inadequate planning, etc. [9]. Global Software Development (GSD) is an attractive domain for software development as it is more economical and integrates the brains of think tanks from all across the globe. Management and security remain challenges for GSD. A hypothetical model shows the positive aspects of GSD using blockchain technology [10]. Software requirements engineering (SRE) is a crucial activity of software development. A detailed investigation of blockchain-acquainted SRE is performed, and a model for SRE is proposed for a blockchain infrastructure [11]. A framework is proposed for Distributed DevOps, another name for GSD, to improve the different development processes using Distributed DevOps [12]. There can be numerous advantages to having a blockchain-based architecture for software effort estimation. Some of the core advantages are in Figure 3.

This research work utilizes the essence of blockchain concerning software estimation for software developed using the Agile software development (ASD) methodology and proposes a model. The implications of the proposed system will not be specific to the effort and time estimation tasks of software project management, but when the proposed model comes into implementation, in a few months, the consortium members will upload more datasets. Different software developers for their estimation problems will use this proposed system, and the datasets will grow with time. The information on more projects will help in improving overall estimation accuracy. At a later stage, the deployed system will function for rewarding bonuses to the practitioners for task management.

There is consensus on the need for blockchain-based software development [13]. Similarly, in a concept paper by Beller and Hejderup [14], the authors published and formulated the term "Blockchain-based Software Engineering," introducing the concept of solving software engineering problems using blockchain. Suggestions for using ML techniques with blockchain-based effort estimation for future work [15].

ASD has secured a high degree of importance in the research community since its publication in 2001 of the Agile Manifesto and its 12 principles [15]. It is important to note that even before what people know as Agile today, the practices and methods used in Agile existed long before. The primary reason for creating the Manifesto was to identify similarities and propose a different approach to software development processes used for the last 40 years [15]. "Agile" refers to various approaches, techniques, methods, and practices that follow the values and principles defined in the Manifesto [16]. ASD is considered an alternative to more traditional approaches, such as waterfall, incremental, or evolutionary, in which predictability, planning, coding, and reusability are the key elements for the efficient development of software [17]. Incremental development models [18], which are the essence of ASD, promote the concept of rapid response to changes and further promote the ideas of customer satisfaction in time, continuous delivery, and minimal planning through informal methods [19].

Recently, there has been a trend of utilizing ML techniques in software effort estimation. ML-based models have provided better software effort predictions than traditional models. The methods based on ML have increased the precision element in cost and effort estimation [20].

Software effort needs data and the analysis of historical data and patterns. Regression models can effectively predict software effort and provide insight into cost [21]. In the literature, the use of ML methods is quite evident [22–24]. Going into detail of the ML methods, the evident ones are regression [25–27], artificial neural network (ANN) [28–31], and ensemble-based techniques [22, 32, 33].

In the SLR by Khan et al. [29], there is a clear indication that ML method application in software effort estimation has increased from the year 2000 onward, and the use of non-ML methods is almost negligible for acquiring good results in effort estimation. Besides this, two SLRs [34, 35] have reported accurately comparing ML and non-ML methods. The two SLRs suggest that ML methods perform better than non-ML methods. Ali and Gravino [36] conducted an SLR on ML methods, concluding that ML methods outperform non-ML methods. They further concluded that ANN and SVM performed better than other ML approaches.

The basic idea of analogy-based effort estimation (ABEE) is estimating a new project through a previous project previously developed by the same organization or some other organization. In the case of other organizations using previous projects as a base, the case will be of industrial data usage instead of historical data. The use of project data does not imply ABEE, as the use of ABEE between different iterations is not evident. The founders of ABEE were Shepperd and Schofield [35], as they proposed this non-algorithm model for the effort estimation of software. As per McConnell [1], ABEE has three constraints for achieving the best possible accuracy. First, the difference between the base and the software under the estimation should be minimal. The second constraint is that both the software should use the same programing language, and the third constraint is that the type of the software under the estimation should not differ from the type of the base software. Taking

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

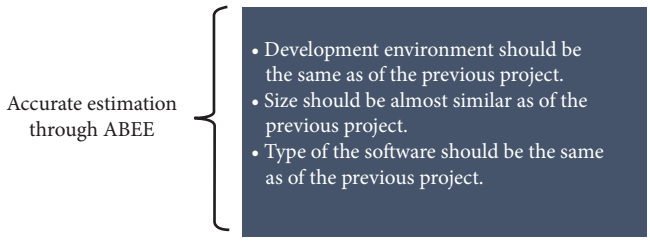

FIGURE 4: Constraints of ABEE to achieve an accurate estimate.

care of these three constraints will achieve an accurate estimate through ABEE, as shown in Figure 4. This means that an ongoing development software's size should not vary much from the software from which it is calculated. The use of a software C language cannot estimate software under-going development in Java or Python. Finally, a system software during development cannot use an application software data for estimating.

The main contributions of the proposed model for the software estimation are as follows:

- This manuscript provides a detailed overview of software effort estimation for ASD that utilizes ML techniques and an overview of blockchain technology for software project management, especially software estimation.
- The integration of ML algorithms with a simulated blockchain technology for software estimation makes this work completely novel.
- A working user application on simulated blockchain technology that takes inputs from the user for user stories, monthly salary, and estimated cost, and then predicts the estimated time for the software using ML techniques is developed. The authors' previous work, based on standard deviation, is also applied for improved software prediction [37].

Hence, the formal definition of the research problem is *"To establish a model for software estimation through analogy-based techniques for ASD using Blockchain technology and machine learning algorithms for better prediction of software estimates."*

The structure of the rest of the article includes the next section provides the related work, which focuses on estimating effort for ASD, including ML techniques and ABEE. It also covers the involvement of blockchain technology in software project management and software effort estimation. Section 3 provides the proposed software estimation model incorporating ML techniques. Section 4 discusses how the ASD model using ML techniques integrates with blockchain technology. Section 5 gives the results with the discussions, and finally, Section 6 concludes the article with directions for future research.

2. Related Work

This section provides a discussion on software estimation, and it incorporates the use of ML techniques and ABEE for software effort estimation. This section also discusses the use of blockchain technology for software project management and software estimation. As the proposed model works on blockchain technology, this section comprises two subsections. The first subsection discusses effort estimation in its current state of the art, and the other subsection discusses the use of blockchain for software project management and estimation. The two main research questions the current research work addresses are as follows:

RQ1: How effectively can ML algorithms predict for software estimation scenarios?

RQ2: How can the integration of blockchain technology enhance the traceability and accountability of project estimation?

2.1. Software Estimation Through ML and ABEE. The core focus of this subsection remains on the software estimation work that incorporates ML and ABEE.

A Bayesian network model for effort prediction in ASD. The use of the Bayesian network is much less in the literature. Even if a few are in place, they lack the validation aspect or have validation for only a few projects. Initially, the model was tested for five output intervals, but then the output intervals were increased without affecting the accuracy [38]. To improve the process of planning poker for product backlog in ASD and to avoid the human errors caused by anchoring effects or overconfidence that may lead to underestimation, an artificial intelligence (AI)-based method is used to reduce the effects of human error factors. Due to influence avoidance, 90% accuracy is expected in the performance [39]. The genetic algorithm (GA) is used to help find the best set of classical case-based reasoning (CBR) parameters, including feature selection, feature weighting, similarity measures, and k number of nearest neighbors. The proposed CBR-GA model validates that the GA algorithm helped in the best combination of CBR parameters, leading to overall accuracy. Validation was performed through evaluation metrics, and the results are promising in accuracy [40]. A work uses decision tree, random forest (RF), and Adaboost to measure project completion in the number of days and Pakistani rupees. The three techniques' bootstrap aggregation (bagging) ensemble provides the highest accuracy, and better project classification helped improve the estimates [41]. A systematic literature review (SLR) identifies the issues related to story-based effort estimation in ASD. The SLR identifies the problems and proposes possible solutions to them. The findings show that recognizing the challenges of task complexity, team dynamics, and requirement engineering can enhance the success rate of ASD [42].

A project selection technique for ABE (PSABE) helps to overcome the criticisms of large memory usage, which reduces the whole project base into a small subset that consists only of representative projects. They combined PSABE with the feature weighting to form FWPSABE to improve ABE further. The results validate their model and conclude that project selection helps improve analogy-based models [43]. A new technique that relies on reasoning by analogy, fuzzy logic, and linguistic quantifiers for estimating effort. They conclude that a fuzzy logic-based model is good if unclear or inaccurate information is considered [44]. For ABEE, the adaptation based on linear

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

regression (LR) outperforms all the other techniques used [45]. Considering ensemble techniques, the research proves by experimenting with seven different datasets that the ensemble technique based on fuzzy analogy outperforms the classical analogy ensemble [46]. The *S* membership function is used to select the proper set of projects and lessen the errors of analogy estimates [47]. Hierarchical clustering techniques are utilized to identify similar projects, and hierarchical clustering produces a range for *k* through various cluster quality criteria [48]. Instead of using all historical data for COCOMO, some recently completed projects of shorter duration will yield more accurate results, and using *k*-nearest neighbors will help produce accurate results for ABEE [49]. Bayesian optimization techniques were used for effort adaptation in 13 standard datasets, and a conclusion was made that the model, which integrates the gradient boosting machine algorithm, will outperform all the other techniques [50].

An investigation for the learning evolution model-based algorithm to optimize the feature weighting, and a new method was proposed. Mean magnitude of relative error (MMRE), PRED (0.25) were used as evaluation metrics, and the proposed technique brought a noticeable improvement in ABEE for calculating the cost [51]. Fuzzy logic with approximate reasoning theory is used to handle imprecision and uncertainty. The initial proposed model was the fuzzy analogy-based software effort estimation (FASEE) model. To further solve the issues of low quality of data and uncertainty in FASEE, an improved model was proposed and named consistent FASEE (CFASEE) [52]. A model prototype is prepared and named EffortEst to overcome ABEE's shortcomings. The work demonstrates that EffortEst provides the best estimation with minimal human intervention [53]. A framework based on CBR to consider functional and nonfunctional requirements with their domain properties. Thirty-six students' projects were used to test the framework, and the difference between the actual and estimated effort was within 10% [54]. Applying linguistics instead of numerical values in ABEE can give much better accuracy. The framework was tested on the popular International Software Benchmarking Standards Group (ISBSG) dataset [55]. Table 1 provides a critical summary of the related work for the subsection of Software Estimation through ML and ABEE. Table 1 consists of seven columns, which are reference, method/technique, application, strengths, and weaknesses. Research gaps and contributions of the proposed model to handle the gaps.

In the domain of software estimation, there is a wide use of ML techniques. The reported studies have focused on MMRE and prediction at 25% (PRED) for evaluating the proposed method. This research work focuses on the use of four ML techniques that are trained on the nearest neighbors identified using KNN. This work utilizes a specific ASD dataset; on top of this, the complete prediction system in built on a simulated blockchain technology, and it further allows the users to use the standard deviation techniques to commit the estimate to the customer with better confidence levels.

2.2. Blockchain in Software Project Management and Software Estimation. The famous and unknown Satoshi Nakamoto, who proposed that this system send payments directly from one party to another, initially proposed blockchain. Since that day, the proposed framework's applications have been immense and are now used in many domains [56], a few examples are [57–60]. Ahmed et al. [61] experimented using a blockchain-based software effort estimation methodology on the expert opinions of 52 organizations. The case study results conclude that a lack of historical data, experts, and biases in a group prevents the organizations from performing estimation activities more effectively. It also suggests that the estimation of blockchain methods is more efficient than traditional methods [61]. Different methods have address the communication and collaboration issues, but they have always lacked security. Sorting the concerns of trust, coordination, transparency, etc., by TestingPlus, which uses blockchain technology for acceptance testing and payments. It also utilized the smart contracts of the private Ethereum blockchain [62].

Software development is incomplete without audits. Although automated tools exist to audit software work products, daily fraud cases remain. The work proposes a blockchain-enabled framework, SSFTA, to aid software auditors in conducting a transparent and effective audit process [63]. With the increase in the usage of GSD, which is beneficial for the overall software industry, there are still barriers to trust, communication, cultures, and time zones. Therefore, a distributed project management framework is proposed to overcome the mentioned barriers [64].

Regarding ASD storyboarding and software development contracts, the major trade-off is stakeholders' confidence. A model that enables initiation of automatic payouts and penalties or grants based on performance. This proposed model helps the partners to develop IT projects [62] successfully. A blockchain-based system for the software process improvement (SPI) to avoid the dominance of being the central authority in SPI, which results in reliance, certification, and high costs. The conclusion is that blockchain can be an excellent substitute for central SPI and help small and medium organizations spend much less money [65]. ChainAgile uses a private Ethereum blockchain to facilitate smart contracts use for ASD from the GSD point of view. The experimentation results show that this approach helps improve security, transparency, communication, coordination, etc., in Scrum Agile distributed software development (SADSD) [66].

A blockchain-based framework to improve DevOps's transparency, traceability, and security problems. To show the effectiveness of the suggested framework, performance results are presented that validate the improvement in the development processes [12]. The integration of blockchain technology to improve the stability and performance of critical chain project management (CCPM) schedules. The research introduces a framework for implementing blockchain-based CCPM buffer monitoring while utilizing the design science research methodology through six research stages: problem identification, literature review, brainstorming, system design, testing, and evaluation. The evaluation on regression analysis and a questionnaire-based survey to assess the prototype system. The findings proposed that the blockchain-based system has the potential for real-time updates, early risk detection, and enhanced collaboration, which will benefit the project

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 1: Critical summary of the work on the use of machine learning in software effort estimation.

Reference	Method/technique	Application	Strengths	Weaknesses	Research gap	Contributions of the proposed model to handle the gaps
[38]	Bayesian network	Agile software development (ASD) effort prediction	Increased output intervals without accuracy loss	Limited validation with few projects	The use of Bayesian network mainly relies on historical data, real-time adaptability is missing	The proposed model of this research work utilizes a hybrid machine learning approach that incorporates real-time data from blockchain technology, enhancing the adaptability and accuracy of software estimations
[39]	AI-based method	Planning poker in ASD	Reduces human error factors, and 90% accuracy is expected	Lacks further empirical validation	Effort estimation only through the product backlog may result in overlooking other aspects of the software	This research work integrates data from multiple sources, because of the presence of blockchain
[40]	CBR and GA (case-based reasoning and genetic algorithm)	Software project effort estimation	Optimized CBR parameters, promising accuracy	Tested only on the PROMISE dataset	The use of genetic algorithms for case-based estimation may not fully use the potential of machine learning in dynamic environments	This research work uses the hybrid approach that will help in better predictions for the estimation cases
[41]	Decision tree, random forest, adaboost	Effort/cost estimation in ASD	Bagging ensemble increases accuracy	Limited to ASD and a small dataset	This study utilizes decision tree techniques but may lack the depth of analysis provided by models that are more complex, where models robustness can suffer	The proposed model uses advance machine learning models and uses the hybrid mechanism
[43]	Project selection for analogy-based estimation and feature weighting	Analogy-based estimation	Reduces memory requirements and computation cost	Lacks extensive cross-domain validation	This study does not include a mechanism for handling uncertainty and does not provide cross-domain validation	The proposed model's idea of a private blockchain-based consortium integrates multiple datasets with the hybrid machine learning approach that can gracefully handle the issue of uncertainty. The use of blockchain technology is also novel, with machine learning
[44]	Fuzzy logic, analogy	Effort estimation	Handles unclear and inaccurate data in a good way	Requires categorical/numerical data representation	This study discusses the idea of using soft computing skills for software cost estimation. There is an expectation of high variance if only the proposed techniques involve modern software development methodologies like Agile	The proposed model is for Agile software development and can even accommodate all iterative-based software development methodologies
[45]	Linear regression adaptation	Analogy-based effort estimation	Outperforms other techniques	There will be adaptation issues to complex project datasets	This study only gives a solution of adaptation and may fail to adapt to complex datasets	The idea of the consortium on a private blockchain of the proposed system will evolve to handle all new kinds of datasets

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 1: Continued.

Reference	Method/technique	Application	Strengths	Weaknesses	Research gap	Contributions of the proposed model to handle the gaps
[46]	Ensemble (fuzzy analogy)	Software effort estimation	Outperforms classical analogy-based models	Requires additional datasets for validation	This study is better than classical-analogy model, but more validation is required on other datasets	The proposed system's idea of multiple datasets on blockchain technology will provide better validation
[47]	*S* membership function	Analogy-based estimates	Reduces errors in estimates	Focused only on analogy methods	The focus of this research work is minimizing the analogy errors. There is consideration of a very specific domain in the research work	The proposed system implements a complete software estimation solution. The realization of the system will enhance the proposed system into full-fledged software project management tool
[48]	Hierarchical clustering	Effort estimation	Identifies similar projects, improves clustering accuracy	Limited to hierarchical clustering approach	This research work is limited to hierarchical clustering and will not be able to provide scalability	The use of blockchain in the proposed model solves the scalability issue
[49]	K-nearest neighbors	COCOMO effort estimation	Uses recently completed projects for higher accuracy	Limited scope, only focuses on shorter duration projects	The historical data in the research work is limited to the COCOMO model only. The COCOMO model has serious issues of subjectivity. The scope is also limited to shorter duration projects	As the proposed model uses machine learning, it solves the problems of subjectivity as well as is capable enough to handle projects of all durations
[51]	Learning evolution model	Feature weighting for ABEE	Significant improvement in accuracy on two datasets	Limited to Desharnais and Maxwell datasets. Validation lacks for datasets for ASD	Only validates to two different datasets, which are very old	The proposed system covers modern datasets for ASD
[52]	Fuzzy logic (CFASEE)	Software effort estimation	Better results on 13 datasets, handles imprecision	Focus was only on fuzzy logic models	Only supports fuzzy logic-based model	This research work uses hybrid mechanism for software estimations. The domain is much bigger in this regard
[54]	Case-based reasoning framework	Software effort estimation	Functional and nonfunctional requirements included, 10% difference between actual and estimated	Tested only on student projects	The testing of the research is only on students' projects	The blockchain technology-based system, using multiple datasets, surely handles software estimation cases, which are versatile
[55]	Linguistic approach	ABEE	Better accuracy with linguistic input	Providing linguistic inputs requires more of effort at the time of estimating the project	This research work requires the use of linguistic inputs at the time of software estimation, which is not always easy to provide	The proposed system simply solves this issue

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 2: Recommendation of blockchain for software estimation.

Reference	The advantages of blockchain for software estimation
[61]	This work highlighted the idea of using blockchain in the domain of software engineering, and software engineers are putting up their entire rigor to resolve the software development issues. The work demonstrates that better accuracy in software effort estimation can be achieved when blockchain-based system is used for effort estimation. This also recommends the use of machine learning with blockchain technology for future work
[11]	A research work demonstrates the use of blockchain technology in the domain of software requirements engineering. The work shows that the blockchain technology can support the requirement engineers and other practitioners in gathering, validating, and specifying software requirements in a decentralized, secure, trustable, and transparent manner. This finding clarifies that by utilizing blockchain technology, software estimation can be conducted in a more structured and trustworthy environment, which will result in achieving more accurate estimates
[14]	In this research work applied, the use of blockchain to the important software engineering problems. The work demonstrates the use of integration and package management with the integration of blockchain technology. This work, however, suggests that blockchain's decentralized nature can facilitate better tracking and management of software development efforts, which will surely help in improved software estimations
[67]	This work is a systematic literature review that identifies different blockchain applications in software engineering. This SLR shows that blockchain-based requirements engineering will enhance the reliability of the data in the software development projects. This claim regarding blockchain-based software engineering supports the concept that blockchain can provide a more robust framework for software estimation by ensuring that the data used for estimations is reliable and verifiable
[68]	An ASD-based software engineering method for designing of blockchain applications. This work shows a structured approach, which includes proposing software development process for gathering requirements, designing, developing, testing, and finally deploying blockchain applications. This structured approach based on ASD strengthens the need for a system where blockchain technology can help in software estimations

stakeholders [9]. In an SLR related to software cost and effort estimation, there is a discussion of blockchain-based effort estimation as a potential future technique. The research also specifies conducting an SLR in the future to cover the core aspects of software development, especially software effort estimation through blockchain technology [7]. Another hypothetical model is proposed using blockchain technology in the domain of GSD. Interviews with academic researchers and industrial experts to identify the impact of blockchain in GSD. This helped achieve 10 critical GSD areas that blockchain technology can address effectively [10].

The integration of blockchain technology into software estimation cases, whether it is related to software effort or time, has acquired important attention in recent years. Blockchain's unique properties can provide an enhancement to the overall software development process, particularly in the domain of software estimation. Table 2 shows the recommendations for the use of blockchain for the cases of software estimation.

Table 2 mentions all the research studies that propose the use of blockchain for the purpose of software effort estimation. This research work has successfully implemented on those recommendations; as the listed research, studies in Table 2 are just suggesting the use of blockchain technology for software effort estimation. Only in the study [61], an effort estimation model that is based on expert opinions via blockchain highlights the suggestion for the use of ML on blockchain technology for software estimation. This research work has made a practical implementation of their recommendation and made it more concrete by accommodating the hybrid approach based on classification and prediction, and finally, also accommodating the standard deviation technique for better prediction [37].

Most conventional software estimation systems use the architecture of a centralized web, where all data are stored and processed under the control of a single authority, mostly the project manager or organizational database. Such systems are easy to implement and are most familiar, but, at the same time, suffer from critical weaknesses, which include vulnerability to manipulation, lack of transparency and audit trail, and trusted administration. These factors significantly endanger the effort estimation task, especially in an Agile environment or any environment involving multiple stakeholders, where cooperation, traceability, and historical soundness of the estimate are required.

Blockchain technology, unlike centralized is decentralized, transparent, and immune to such failures. It ensures immutability of records, automated rule enforcement through smart contracts, and secure, auditable logs of every single transaction or change to estimation data. This will not just increase trust of both teams and stakeholders but will increase accountability, version history, and compliance across the software estimation lifecycle. Table 3 illustrates the major points of blockchain vs. the centralized system, with special focus is given on how each contributes to a more accurate, trustworthy, and scalable software estimation framework.

The following section provides the estimation model on which the ML techniques were incorporated.

3. Software Estimation Model for ASD

Ziauddin and Zia [75] proposed an effort estimation model for ASD. This model uses data from 21 software projects of Pakistani software houses. The dataset is widely used in software effort estimation research and has many citations [41, 76–78]. Therefore, the proposed model used this dataset after

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 3: Advantages of blockchain for software estimation compared to a centralized web-based system.

Feature/benefit	Blockchain-based system	Centralized system	How it supports software effort estimation	Key references
Immutability	☑Tamper-proof, once written, cannot be changed	☒Estimations or logs can be altered or deleted	Ensures historical effort estimates and predictions are not retroactively modified, enabling traceability and audit	[69]
Decentralization	☑Distributed storage and consensus, no single point of control	☒Entire control lies with a central authority	Prevents manipulation or bias from a single party; supports trust in multiteam or multiorganization projects	[70]
Transparency and auditability	☑Fully traceable history of all estimation entries, visible to all stakeholders	☒Limited visibility; only admins can trace changes	All parties (developers, PMs, clients) can verify how estimates evolved and whether process compliance was followed	[71]
Security	☑Cryptographically secured transactions, consensus-driven validation	☒Vulnerable to internal threats or data breaches	Effort estimates and sensitive inputs (e.g., cost drivers) are protected from tampering and unauthorized access	[72]
Trustworthiness	☑Trust is embedded in protocol; no need to rely on a central admin	☒Users must trust centralized authority	Teams can rely on shared predictions without requiring trust in a controlling party, improving collaboration	[73]
Smart contracts/automation	☑Rules enforced automatically (e.g., lock estimates postsubmission, notify on changes)	☒Manual intervention needed; rules often bypassed or overlooked	Automates validation of inputs and consistency checks (e.g., timelines, developer availability), reducing manual errors	[74]

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

augmenting it. The proposed model provides a graphical user interface (GUI)-based system developed in Python that predicts the estimated time for project completion. After applying the ML techniques for generating predictions, a comparison of the predicted results of the proposed and Ziauddin models is performed through mean relative error (MRE), MMRE, and prediction at level 25% (Pred (25)) evaluation metrics.

In Section 4, the author's previous research on standard deviation-based techniques is also incorporated for better prediction [37]. This simulated blockchain-based estimation model will help collaboration among the stakeholders and will evolve into a more mature software estimation framework.

For the prediction of software completion time, the proposed model utilizes the augmented version of the dataset for projects from software houses in Pakistan. Three inputs of effort in user stories, the monthly salary, and the estimated cost help in making new predictions. Although the original model [75] gave an estimated cost as an output, it is used as an input in the proposed model. Using the input parameters, the output of estimated time using four ML techniques: RF, LR, support vector regression (SVR), and extreme gradient boosting (XGBoost). This model also used ABEE based on nearest neighbors; all ML algorithms use the nearest neighbors.

The hybrid approach initially using KNN and then the predicting with LR, RF, SVR, and XGBoost ensures that the model trains on the most relevant subset of data for each prediction. Instead of training models on the entire dataset, the KNN approach selects local neighbors of similar projects, which train the models for making more targeted predictions. This allows accurate predictions, where the models learn from a subset of the data that is more appropriate and related to the input features. LR assumes a linear relationship between the input features and the target variables; the core reason for using this technique was on the grounds of its simplicity and its ability to serve as a baseline model. In RF, as there are multiple decision trees and it is an ensemble-based technique, it is very effective in capturing nonlinear relationships between input features and target variables. In software estimation, the relationship between size and effort or between size and time does not follow the economies of scale [1]. The use of RF will help in providing better prediction for such cases where linear relationships do not exist clearly, and it is also robust against overfitting and also supports larger datasets. SVR utilizes the kernel trick technique to map data into higher-dimensional spaces, which allows SVR to capture complex patterns that linear models can miss, but it can struggle with larger datasets. XGBoost, a gradient boosting ensemble method, further enhances predictive power by iteratively optimizing decision trees to minimize errors, effectively handling nonlinear relationships and scaling efficiently with larger datasets. To accommodate for nonlinear relationships handling, the model uses RF, SVR, and XGBoost techniques; considering three techniques, SVR is ideal for smaller datasets, while RF and XGBoost can cater for larger datasets. Another reason to predict using four different techniques is also that the estimator will look for convergence among the four techniques. If three or all four predictions are converging with each other, this will give a stronger element of confidence to the estimator about the prediction.

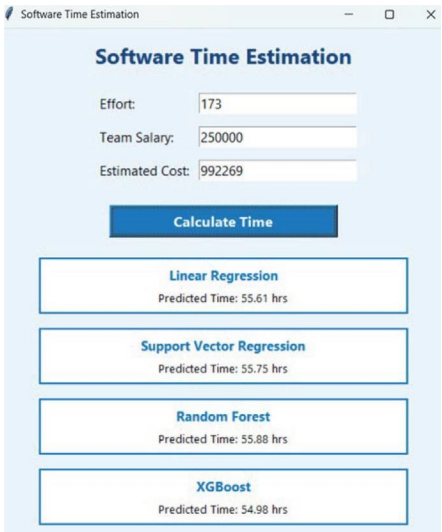

FIGURE 5: GUI of time prediction software.

This work estimates the completion time and effort for a project using ML techniques. The project time predictions are using supervised ML techniques based on historical data. Figure 5 depicts the GUI.

For time estimation, as the dataset comprises only 21 projects, the dataset augmentation on a factor of five takes the projects to 126. The main reason of using this dataset was twofold; first, this dataset is specific to ASD, and second, this dataset comprises of actual completion time of the projects. This field of actual completion time helped in authenticating the proposed model by using the evaluation metrics. Whereas the main reasons for augmenting the dataset are that the model learns more generalized patterns rather than memorizing the limited examples. This augmentation also helps in mitigating overfitting, and increased dataset size correlates with better prediction accuracy. The process of augmentation applies some controlled transformations to the original dataset, such as introducing small and random noise to the input features like team salary, effort, and estimated cost. This method of augmentation minimized the risk of creating artificial noise or biasness. This expanding of the dataset helped in training and testing the predictive models on a wider range of conditions. This process helps in enhancing the generalizability and robustness of the results. The key Python functions used for augmentation were pandas.DataFrame.copy(), for creating the copy of the original dataset for augmentation, numpy.random.normal(), for adding random noise to the numerical features based on mean and standard deviation. Other functions for the augmentation process were pandas.concat(), for combining the original and augmented data into a single dataset, and finally numpy.tile(), which was used for repeating the original dataset for structured augmentation.

The proposed model also uses a dataset for effort estimation, which has 136 records about ASD projects, created expressly to support effort estimation through ML techniques. The dataset features contain fields for team composition, project nature, and technological dependencies, as well as hours of person-hour effort as the target variable. The dataset was

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

developed in a research work that uses an ensemble method for software effort estimation and is specific for Agile software [79]. The selection of the features, size, complexity, and number of tasks (noftasks) as the input features (X) based on their strong correlation with effort and their relevance in software engineering literature. Size is important because it directly reflects the scope of the project; LOC or FPs helps in measuring size. Complexity was included as it captures the technical difficulty of the project, which significantly influences the time and resources needed, especially for intricate tasks. The inclusion of the Noftasks is on the basis that the Noftasks influences the workload and coordination effort, making it a critical determinant of total effort.

Table 4 shows the estimated time results of all four ML techniques used in this research. Table 4 also shows the three input parameters for all 126 software and the estimated time through the originally proposed model by Ziauddin and Zia [75]. Table 4 shows the results of the basic estimated time of the original model under the field named estimated time. The other four are the estimated times for RF, LR, SVR, and XGBoost. It is important to understand that the estimated time from the four ML techniques also involves the nearest neighbor calculation for better prediction. Section 5 discusses the actual MRE calculations for all four models, along with the original model.

Table 5 shows the estimated effort results of all four ML techniques used in this research. Table 5 also shows the three input parameters for all 136 software and the estimated effort through the four ML models. Consideration of the effort fields of the dataset as the actual effort because a base model does not exist for comparison of effort. The actual MRE calculations for all four models for the effort prediction come in Section 5. Tables 4 and 5 both contain the results of the first six and last five projects, respectively, for the estimation of time and effort, respectively. The reason to display lesser records is in order to save the space. The complete implementation codes, the real datasets, and the results of the time and effort estimation are available on GitHub, along with the link in Section 5.

As shown in Figure 5, for the prediction of time, the GUI asks for the users input parameters. The nearest neighbor function finds projects similar to the user's inputs. Once the neighbors are identified, the four ML techniques are trained on the identified number of neighbors, which is set to $k = 7$ for all experiments in this research work.

Formulas (1)–(4) provide prediction formulae for the RF, LR, SVR, and XGBoost, respectively.

$$\widehat{Y}(X) = \frac{1}{N}\sum_{i=1}^{N} T_i(X). \tag{1}$$

RF combines multiple decision trees for better prediction to control overfitting. In the specific case of predicting time or effort, each tree outputs a prediction. The final prediction is the average of all the predictions. In Formula (1), T_X denotes the prediction of the i-th tree for the input vectors X and N, which is the total number of trees in the forest.

LR fits a linear model to the training data. For a dataset with n samples and m features, the model predicts the target variable y based on linear combinations of the features X.

$$\widehat{Y}i = w^T X_i + b. \tag{2}$$

LR seeks to find coefficients "w" such that the predicted values $\widehat{Y}i$ approximate Y_i. SVR performs regression by finding a hyperplane in a high-dimensional space that best fits the data. It uses a loss function that penalizes predictions outside a specified margin from the target values. SVR aims to find a function $f_{(x)}$ that predicts continuous target values y based on input features X.

$$\widehat{Y}(X) = W^T \varnothing(X) + b. \tag{3}$$

In Formula (3) W, the weight factor determines each feature's importance. $\varnothing(X)$, is the feature mapping function that transforms input X into a higher dimensional space, and b is the bias term that adjusts the offset of the regression plane.

XGBoost is a decision-tree-based ensemble ML algorithm that uses a gradient boosting framework. It builds additive models in a forward stage-wise fashion by optimizing a differentiable loss function. Each successive tree trains to correct the residuals (errors) of the combined ensemble of previous trees. The prediction from the XGBoost model, given in Formula (4).

$$\widehat{Y}(X) = \sum_{(k=1)}^{K} f_k(X),\ f_k \in F. \tag{4}$$

Here, $f_k(X)$ denotes the output of the k-th regression tree in the model, and K is the total number of trees. The space F represents all possible regression trees. XGBoost improves both bias and variance through regularization and has built-in mechanisms to handle missing data and control overfitting, making it particularly effective for structured datasets.

Figure 6 shows the complete flowchart of the code in Python for the time estimation prediction. The GUI shown in Figure 5, it also contain the input parameters and then displays the outputs for the four ML models.

This next section discusses the details of blockchain technology and provides the complete implementation mechanism for blockchain technology using the ML-based software estimation model.

4. Integration of Blockchain Technology in Our Proposed Model

Related work sections discuss the advantages of having a blockchain architecture for the project management of software. Here, there is a discussion on the advantages of the time estimation system for ASD on blockchain technology. It will be an immutable ledger comprising a time estimation result for each transaction. This immutability will help maintain trust and accountability in activities confining to software project management. As blockchain is decentralized, it will reduce the risk of a single point of failure. The use of cryptographic techniques will assure the security of data. Smart contracts can be widely used in the system. Initially, after the validations of the software estimate when it adds to the blockchain technology, the stakeholders receive the notifications. New calculations add to the

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 4: Estimated time in person-hours of the original model and three machine learning techniques.

Project ID	Effort	Team salary	Estimated cost	Estimated time	Estimated time random forest	Estimated time linear regression	Estimated time support vector	Estimated time XGBoost
1	156	230,000	1,023,207.14	58	62.48	63.31	63.1	62.39
2	202	260,000	1,680,663.89	81	91.69	91.77	63.1	92.10
3	173	250,000	992,269.51	52	55.57	55.56	55.5	55.20
4	331	300,000	2,002,767.22	87	85.63	85.99	86.9	84.66
5	124	300,000	676,081.32	29	32.30	32.48	33.1	32.42
6	339	400,000	2,895,132.85	95	91.02	90.74	90.75	90.79
–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–
122	112.7	250,000.8	770,859.5	40.9	39.61	39.20	39.7	39.16
123	140.6	250,001.1	961,867.5	49.6	51.26	50.94	51.8	50.30
124	212.5	250,001.5	1,453,033	76.2	80.32	80.23	80.5	79.50
125	136.7	219,998.8	854,347.4	50.3	55.52	55.26	55.2	55.20
126	91.7	219,999.7	567,483.2	34.2	35.09	34.97	35.05	35.06

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 5: Estimated effort of the four ML models.

Project	Size	Complexity	No. of tasks	Actual effort	LR_Prediction	SVR_Prediction	RF_Prediction	XGB_Prediction
1	3	2	2	3	2.5	2.1	2.4435	2.499166
2	4	4	4	8	5	5	5	5
3	1	1	4	1	1	1	1	1
4	4	4	3	5	5.75	5.1	5.669571	5.749512
5	3	3	4	3	2.666667	2.9	2.6955	2.667208
6	1	1	4	1	1	1	1	1
–	—	—	—	–	–	–	–	–
–	—	—	—	–	–	–	–	–
132	5	5	1	13	6.3	5.51	5.38	5.0
133	3	3	3	3	2.66	2.9	2.66	2.66
134	1	2	3	2	2.21	2.17	2.02	1.99
135	5	5	3	5	9.5	7.79	8.93	9.0
136	1	1	3	1	1.28	1.1	1.26	1.28

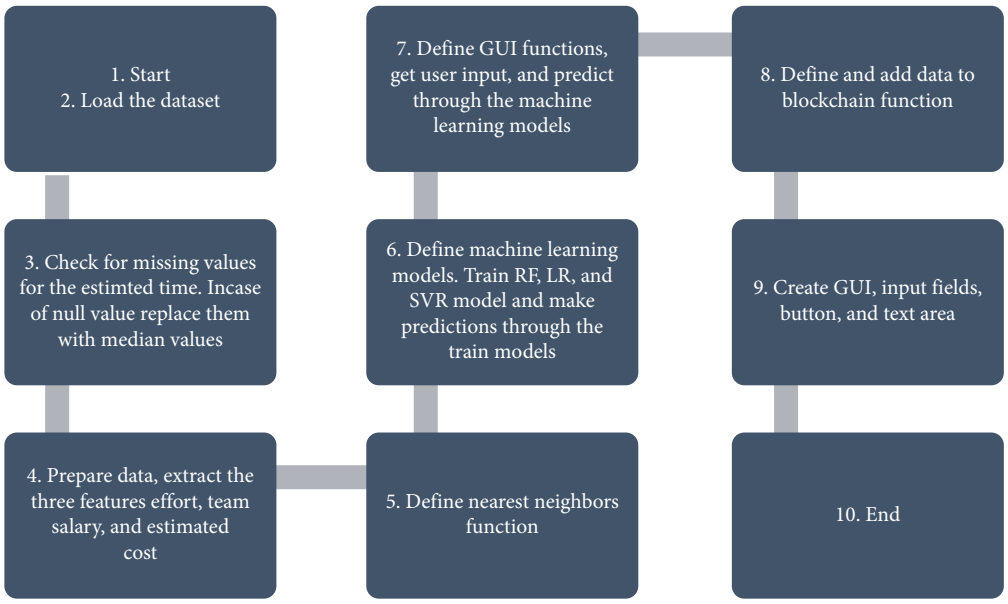

Figure 6: Flowchart for the prediction of completion time.

dataset, which gives the concept of scalability of the software estimation system.

A group of software houses can start this already tested system by forming a consortium, which may collaborate to form a private blockchain. By implementing a private blockchain, the software houses can create a secure and transparent system for sharing software time or effort estimation data. This will ultimately help improve project planning accuracy and enhance client trust.

Eventually, when implemented, this blockchain-based software estimation system will evolve. The more usage will create more benefits when transactions related to the software estimation come to the blockchain. The loyal and old stakeholders can have some additional benefits compared to those who join later. Figure 7 shows the complete blockchain architecture of the software estimation system.

Figure 7 shows the system's major activities. User requests to join the private blockchain consortium. After the verification and approval of the request, the user receives the credentials, like cryptographic keys, to participate in the blockchain network. Then, the user prepares the estimation case. In the system, users enter parameters like effort in user stories, the monthly team salary, and the estimated cost. The user encrypts and submits the estimation case as a new transaction on the blockchain network. The nodes in the private network receive the transaction, and a consensus mechanism like proof of authority validates the transaction on some predefined rules. After that, the complete block of the validated transaction, adds to the blockchain's ledger. The dataset for estimation, including all the previous and current projects, is stored as immutable records on the blockchain. The new estimation case contributes to the growing dataset on the blockchain architecture. Data are

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

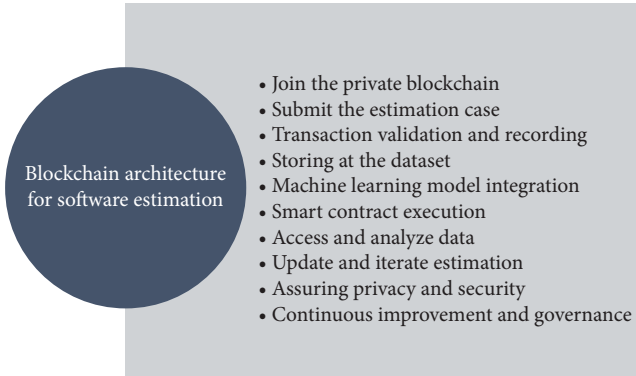

FIGURE 7: The architecture of the blockchain-based software estimation system.

stored with cryptographic hashing to ensure integrity within the blockchain architecture. The ML model comes into play when the estimation case triggers the system, and relevant features are extraction of the submitted case takes place. The features are preprocessed using algorithms like RF, LR, SVR, and XGBoost. The model initially trains on the historical data stored in the blockchain, and then the prediction for the estimated time or effort for the new case uses the trained model.

Smart contracts can help to validate the estimate's accuracy against some predefined thresholds before finalizing and recording the estimated time. When a client enters the details of a new software project for estimation, and the values for the estimates using ML techniques are calculated, the smart contract takes charge of the entire estimation process by providing immutability. It helps increase the stakeholders' trust and promotes fairness and accountability. Figure 8 shows how the user interacts with the software estimation system and how the smart contract execution takes place.

A sample smart contract, as shown in Figure 9, helps in decentralized project time estimation by allowing clients to input project details like effort in terms of story points, the monthly salary of the team on a particular project, and the estimated cost of the project by using the model of Ziauddin and Zia [75]. This software time estimation uses blockchain technology. The contract securely stores the inputs the user provides and uses an ML model to estimate project time based on historical data. When the contract receives the client inputs, it records these details. It depicts estimation results for four ML models, RF, LR, SVR, and XGBoost, which are stored alongside the project details. This contract enables transparent and immutable recording of estimation processes, fostering trust among project stakeholders in a decentralized environment. By leveraging blockchain technology, the smart contract ensures that project estimations are securely stored and retrievable, supporting fair and efficient project management and bidding processes within decentralized applications.

At a later point of time, multiple smart contracts to enhance flexibility and scalability can evolve the entire system. For example, specialized smart contracts for each phase of the software lifecycle, which include requirements gathering, coding, testing, and deployment, can come into action. Each contract would be responsible for specific time predictions for a particular phase of the lifecycle, thereby enabling more detailed and modular estimation. Moreover, hybrid contracts combining traditional prediction models with advanced ML techniques to dynamically adapting to changing project parameters can be a part of the system. Another key development would be the integration of smart contracts that automatically adjust based on historical performance data, thereby improving the accuracy of future predictions. The smart contracts can also handle the payment procedures for different stakeholders when they meet a target.

The proposed model for the prediction of software completion time or effort employs ML, including RF, LR, SVR, and XGBoost, with an additional refinement using the standard deviation technique to improve the reliability of estimates. While the blockchain layer is conceptual in this work, the article discusses how integrating a smart contract can securely store and audit predictions, thereby enhancing transparency and traceability in future implementations as shown in Figure 10.

The consortium members can access the blockchain ledger to view historical estimation cases and datasets. Analysis to improve the accuracy of ML predictions can strengthen the system. With every new estimation case the blockchain ledger updates, the consortium members can also make more strategic decisions related to estimation, like different input criteria for users adding up more datasets for different estimation cases. The immutable nature of blockchain provides data integrity and audit mechanisms. The system goes under a continuous improvement cycle through a consortium governance structure. The preceding subsection shows the practical deployment steps of the proposed model, and then Section 5 discusses the results.

4.1. Practical Deployment Summary. The following points discuss the practical deployment summary of the proposed model:

- Data input and preprocessing: Project data are entered in the system and preprocessed for the ML models.
- Prediction using ML models: The preprocessed data goes to the ML models for the prediction of effort or time.
- Prediction refinement: The predictions can be refined using the standard deviation technique to have more confidence on the predictions.
- Blockchain transaction preparation: A blockchain transaction stores the output.

5. Results and Discussion

This section shares the results of the actual predictions on the Ziauddin and Zia [75]. This includes the calculated results and the MRE calculations for all five cases. Table 5 shows the first MRE for the original model of Ziauddin, and then the MRE for the four ML techniques. The calculation of the MRE uses the actual time field given in the dataset, which helps in calculating the MRE for the original dataset and the four ML techniques of the proposed model.

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

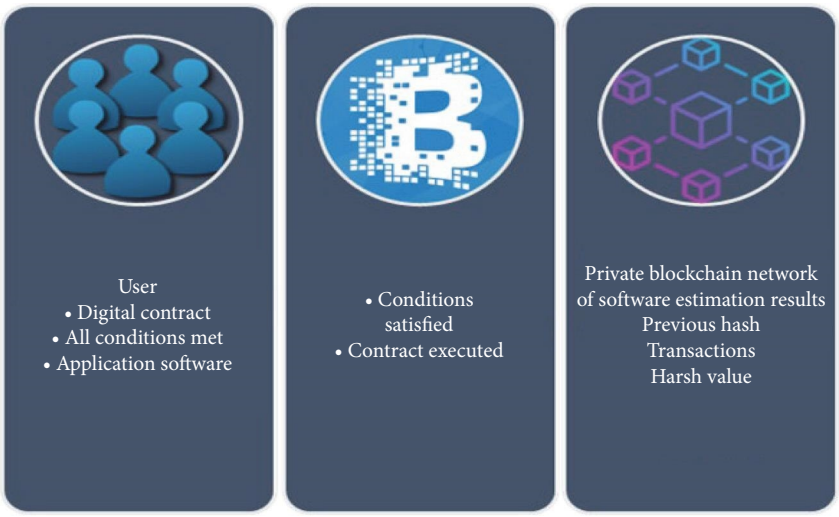

Figure 8: The blockchain-based software estimation system.

```
pragma solidity ^0.8.0;
contract EffortStorage {
   struct EffortEstimation {
      uint effort; uint teamSalary; uint estimatedCost;
      uint linearRegression; uint supportVectorRegression;
      uint randomForest; uint xgBoost;
   }
   EffortEstimation[] public estimations;
   function storeEffortEstimation(
      uint _effort, uint _teamSalary, uint _estimatedCost,
      uint _linearRegression, uint _supportVectorRegression,
      uint _randomForest, uint _xgBoost) public {
      estimations.push(EffortEstimation(
         _effort, _teamSalary, _estimatedCost,
         _linearRegression, _supportVectorRegression,
         _randomForest, _xgBoost));
   }
   function getEffortEstimation(uint index) public view
      returns (uint, uint, uint, uint, uint, uint, uint) {
      EffortEstimation memory e = estimations[index];
      return (e.effort, e.teamSalary, e.estimatedCost,
            e.linearRegression, e.supportVectorRegression,
            e.randomForest, e.xgBoost);
   }
   function getCount() public view returns (uint) { return estimations.length; }
}
```

Figure 9: Pseudo-code for the smart contract of the time estimation system.

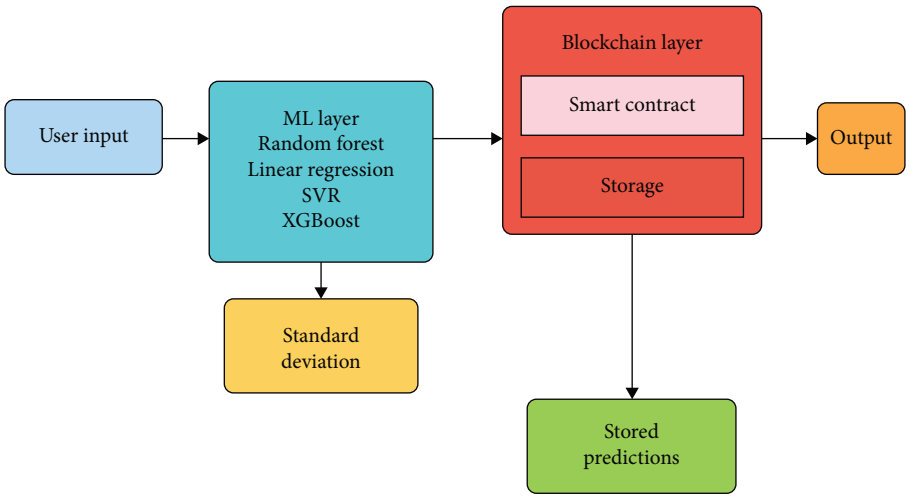

Figure 10: The ML and blockchain layer of the proposed system.

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

This section discusses the platform, implemented code, evaluation criteria, research question, and the details of the datasets before sharing the actual results.

5.1. Platform. The experimentation uses Python as the primary programing language. The GUI for the time estimation uses the Tkinter library, which provided the facility for the user input for parameters on a web-based interface. The experiment utilized a simple Core i5 machine with 16 GB of RAM.

5.2. Implemented Codes. The following modules are included in the code of the time estimation system.

5.2.1. Data Preprocessing. The dataset is initially loaded and then preprocessed to handle the missing values in the target variable, EstimatedTime. StandardScaler for scaling the features of Effort, TeamSalary, and EstimatedCost helps to improve the model performance.

5.2.2. Model Training. The training of the four ML models, LR, RM, SVR, and XGBoost, uses the preprocessed dataset. All models use the similar projects identified through a nearest neighbor approach.

5.2.3. Blockchain Simulation. In order to enhance the creditability and traceability of the predictions, a simulated function is for adding data to the blockchain is included. This function is very important for future audits and accountability in software project management.

5.2.4. User Interaction. As mentioned in the platform section, the GUI uses the Tkinter library. The GUI asks the users to enter the details and displays the prediction results.

5.3. Evaluation Criteria. This research work makes the use of three key metrics for the evaluation of the implemented models:

- MRE: This metric measures the relative error of the prediction compared to the actual completion time.
- MMRE: This metric calculates the mean of the absolute values of the relative errors. It gives a view of the magnitude of the prediction error across a number of projects.
- Pred (25): This metric measures the percentage of predictions whose relative error falls within 25% of the actual value.

5.4. Research Questions. The aim of this study is to address the following research questions. These are the findings for the identified questions.

- RQ1: How effectively can ML algorithms predict for software estimation scenarios?

Answer: In the related work section, numerous research work that employs the ML techniques for software estimation are elaborated and cited. The findings of this research study are that ML algorithms, which perform training on the historical data of ASD, can enhance the accuracy of the completion time predictions. Even they give better results than compared to the original model [75], whose dataset is used for the software time estimation system of this research work.

- RQ2: How can the integration of blockchain technology enhance the traceability and accountability of project estimations?

Answer: Integrating the blockchain technology into such a system provides an immutable and transparent record of project estimates and changes throughout the development process. It helps in increasing the trust among the team members and the clients by ensuring that all project data is securely logged and accessible.

5.5. Dataset

5.5.1. Dataset for the Time Estimation. The original dataset for the time estimation for this study only comprised of 21 projects of ASD [75], so augmentation on a factor of five takes the total project to 126. The selection of five features to be included in the experimentation part of this research work provides a quantitative basis for estimating the time required to complete each project. The actual time feature comes into play for calculating the MRE, MMRE, and Pred (25) of all the models. The explanation of the attributes is as under:

- Effort: This represents the total effort of the project in person-hours. It shows the amount of work put into the project and is a very crucial attribute.
- TeamSalary: This refers to the actual salary paid to the team, which is involved in the project, and is in currency units.
- ActualTime: This talks about the actual time of the project completion and is the number of hours. A very crucial attribute to validate the predictions and using the metrics for evaluation criteria.
- EstimatedTime: This shows the estimated time to complete the project and in hours. These values also provide the baseline for assessing the performance of the different predictive models, including LR, RF, SVR, and XGBoost.
- EstimatedCost: The estimated cost of the project in currency units. It shows the required financial resources which are needed for the implementation of the project.

5.5.2. Dataset for the Effort Estimation. The dataset for the effort estimation prediction for this study comprised of 136 projects [79]. The selection of four features to be included in the experimentation part of this research work provides a quantitative basis for estimating the effort required to complete each project. The explanation of the attributes is as under:

- Size: This denotes the overall size of the project, which could reflect factors like LOC of the FPs.
- Complexity: This represents the complexity level of the project tasks, which can significantly affect the amount of effort needed.

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

- Nooftasks: The Noftasks involved in completing the project.
- Effort: This is the target variable, representing the actual effort in person-hours needed to complete the project.

All the features in this dataset are continuous and numerically encoded, making them suitable for use in ML pipelines. Moreover, the models evaluated include LR, SVR, RF, and XGBoost (XGB), and the validation of results using leave-one-out cross-validation (LOOCV) to ensure robustness in small datasets.

5.6. Results and Discussion. Tables 4 and 5 depict the input parameters for the 126 dataset of time estimation and the 136 projects dataset of the effort estimation, so Tables 6 and 7 do not show them. The last row of Tables 6 and 7 shows the MMRE for all five techniques for the time estimation and four techniques for the effort estimation, respectively. The effort estimation just provides a dataset, and there is no base model, which is why it has the MMRE for the four ML techniques, as the MRE uses the effort field as the actual effort for MRE calculations.

Table 6 shows that the MMRE is the best of the SVR models, which is 1.73% for all 126 projects. It is also important to note that the MMRE of all ML models is better than the original model. This supports the findings of the research manuscript that ML helps in the overall accuracy of the estimation, as the ML models performed much better than the original model. Phenomenal results for the Pred (25) as the original model and the three ML models give 100% count for the Pred (25), meaning all 126 predictions for the four models were within the 25% variance value. Only the XGBoost has a count of 122, which is again very good.

Table 7 shows the estimated predictions of effort from all the four ML models and the MRE, MMRE, and Pred (25) calculations. The MRE uses the effort field of the dataset, considering it as the actual effort of the software.

Table 7 shows the effort predictions and the MRE, MMRE, and Pred (25) for all 136-project cases using the four ML techniques. The average MMRE for all four models is 23.6% with only the MMRE of LR going slightly above 25% at 25.74%, rest for the three other ML models its below 25%. This result is also good, considering the fact that the estimation variance from the actual is below 25% as per [1]. The last column shows the Pred (25) counts of the total 136 projects, which is not as good as for the time estimation case, but considering the fact that a comparison model was not in place, it is considered good. The best Pred (25) count is 88 out of 136 projects, which is of SVR, and the best MMRE is again of SVR at 20.43%.

The MMRE of the original model for the time estimation case is 6.82%. The MMRE of LR, RM, and XGBoost are 2.42%, 2.02% and 3.36%, respectively. The best MMRE is of SVR, which is 1.73%. This shows that SVR performed best than compared to other ML models. All ML model gives the better results than the original model. The complete code, datasets, and readme file for using the software estimation for software completion time and software effort are available at https://github.com/ayublatif/Software-Estimation/tree/main.

The statistical validation for the performance improvement of the proposed model uses the Wilcoxon signed-rank test. This nonparametric test is widely used to compare paired samples when the normality assumption may not hold, making it ideal for evaluating prediction models across the same dataset. In this study, the application of the Wilcoxon test to compare the baseline original model against ML models such as LR, SVR, RF, and XGBoost. The resulting p-values (e.g., LR: 1.74×10^{-14}, SVR: 2.44×10^{-18}) were all significantly less than the standard threshold of 0.05, indicating statistically significant differences in performance. Thus, the results confirm that the proposed models significantly outperform the baseline with high statistical confidence. Table 8 shows the p values for the original model and the four ML techniques, and Figure 11 shows the values on the graph.

The results of the time prediction are already very good, so the use of the standard deviation technique to predict with better confidence is used with the effort estimation predictions. McConnell [1] specified the standard deviation for better prediction in his book. This technique was used in the authors' previous work on the time prediction dataset without applying the augmentation [37]. For the effort, estimation case SVR gives the best results than other ML techniques, and the MMRE is the highest of LR. Considering this, this work uses the standard deviation technique to predict the LR estimates with more confidence levels, as it has the worst MMRE. The technique requires the expected case to use the formula of the expected case, as shown in Formula (5).

$$\text{Expected case} = \frac{\text{Best case} + (4 \times \text{most likely}) + \text{worst case}}{6}. \tag{5}$$

A known way in statistics is to assume that the one-sixth difference between maxima and minima equals one standard deviation. This way of calculating the standard deviation assumes that the maxima include a 99.86% chance of meeting the estimate and the minima hold a 0.135% chance. The standard deviation will be the difference between maxima and minima divided by 6 [1]. The technique requires the best-case and the worst-case estimates for the LR technique to incorporate the standard deviation for the overall 136 projects. For LR estimates, we applied an optimistic adjustment of −7% and a pessimistic adjustment of +10% to generate the best-case and the worst-case scenarios. The choice of these values is empirically on the observed error distribution of the LR predictions. The slightly tighter bounds help control the standard deviation, reduce overestimation bias, and keep the confidence intervals closer to the base LR estimates. This approach balances the need for realistic variance representation while minimizing MMRE at 70%, 80%, and 90% confidence levels, thereby improving the reliability of the refined estimate. The selection of subtracting 7%, which is slightly lower than adding 10% for the pessimistic approach, also supports the idea that there is a limit on how well a project can go, but no limit on how bad a project can go [1]. The calculations make use of the actual prediction as the most likely value. Formula (5) helps in

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 6: Estimated time in person-hours of the original model and four machine learning techniques, along with their MRE, MMRE, and Pred (25).

ID	Estimated time	Actual time	LR	SVR	RF	XGB	Original_MRE	LR_MRE	SVR_MRE	RF_MRE	XGB_MRE
1	58	63	63.31	63.1	62.486	62.390	0.079	0.004994	0.001587	0.008	0.009
2	81	92	91.77	92.05	91.691	92.100	0.119	0.002483	0.000543	0.003	0.001
3	52	56	55.56	55.5	55.57	55.209	0.071	0.007768	0.008	0.007	0.014
4	87	86	85.99	86.9	85.636	84.66	0.011	0.00	0.010465	0.004233	0.015
5	29	32	32.48	33.1	32.308	32.42834	0.093	0.015164	0.034375	0.009625	0.013
6	95	91	90.74	90.75	91.026	90.79	0.043	0.002837	0.002747	0.000286	0.00
–	–	–	–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–	–	–	–
122	40.9	38.4	39.20	39.7	39.612	39.16	0.0651	0.021071	0.033854	0.031562	0.019916
123	49.6	50	50.94	51.8	51.263	50.30	0.008	0.018914	0.036	0.02526	0.006026
124	76.2	79.5	80.23	80.5	80.327	79.50	0.0415	0.009301	0.012579	0.010403	9.70E-05
125	50.3	56.6	55.26	55.2	55.522	55.20	0.111	0.023623	0.024735	0.019046	0.024735
126	34.2	33.8	34.97	35.05	35.09	35.061	0.0118	0.034669	0.036982	0.038166	0.037317
	MMRE	—	—	—	—	—	6.82%	2.42%	1.73%	2.02%	3.36%
	Pred (25) counts	—	—	—	—	—	126	126	126	126	122

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 7: Estimated effort and MRE, MMRE, and Pred (25) calculations of the four machine learning techniques.

Project	Actual effort	LR_Prediction	LR_MRE	SVR_Prediction	SVR_MRE	RF_Prediction	RF_MRE	XGB_Prediction	XGB_MRE
1	3	2.5	0.1666	2.1	0.3	2.44	0.185	2.499	0.16
2	8	5	0.375	5	0.375	5	0.37	5	0.37
3	1	1	0	1	0	1	0	1	0
4	5	5.75	0.15	5.1	0.02	5.669	0.133	5.7	0.14
5	3	2.66	0.111	2.9	0.03	2.695	0.1015	2.6	0.11
6	1	1	0	1	0	1	0	1	0
–	–	–	–	–	–	–	–	–	–
–	–	–	–	–	–	–	–	–	–
132	13	6.3	0.51	5.513	0.575	5.383	0.585	5.0	0.61
133	3	2.666	0.11	2.9	0.0333	2.666	0.111	2.66	0.11
134	2	2.214	0.107	2.174	0.08	2.02	0.01	1.9	0
135	5	9.5	0.9	7.794	0.55	8.93	0.786	9.0	0.80
136	1	1.28	0.28	1.1	0.1	1.265	0.265	1.28	0.28
	MMRE	—	25.74%	—	20.43%	—	24.11%	—	24.19%
	Pred (25) counts	—	83	—	88	—	77	—	79

ietsfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 8: Wilcoxon signed-rank test results (p-values).

Model	p-Value	Interpretation
Original	8.30×10^{-7}	Significant
LR	1.74×10^{-14}	Significant
SVR	2.44×10^{-18}	Significant
RF	5.33×10^{-17}	Significant
XGBoost	4.13×10^{-11}	Significant

generating the expected values for all 136 projects. This approach helped calculate the standard deviation, generating different estimated efforts with different confidence levels. Table 9 shows the confidence level for the standard deviation technique.

These steps of the standard deviation technique are as under:

1. For the calculated time or effort, generate their pessimistic and optimistic values. Historical data usage can help in accomplishing this task.
2. Apply the expected case formula to generate the expected case value for the calculated time or effort.
3. After the calculation of the expected value, use the value for calculating the predictions with different confident cases. Table 7 shows the formulae for calculating the predictions with 70%, 80%, and 90% confidence.
4. Commit the calculated completion time based on your confidence level to the customer. These values with different confident levels will give more confidence to the estimators for committing the completion time to the customer.

Table 10 shows the best case, most likely, the worst case, and expected case estimations of the calculated effort using the LR technique through Formula (5). This exercise helps to get the standard deviations for all the estimation cases. The standard deviation is calculated for all estimated projects by dividing the difference between maxima and minima by 6 [1].

Table 11 depicts the LR-based estimated effort with 70%, 80%, and 90% confidence levels using the calculations given in Table 9.

Table 12 shows the MRE and MMRE calculations for four cases, the MRE and MMRE for the initial LR-based calculation initially, the MREs and MMREs for the LR estimation with 70%, 80%, and 90% confidence levels, respectively. It is vigilant from Table 12 that there is an improvement in the MMRE at all three confidence levels than compared to the MMRE that was originally calculated. Figure 12 also depicts the MMRE at all confidence levels.

The core purpose of this research is to propose a blockchain-based software estimation system that uses ML techniques to predict software estimates. This prediction accommodates a simulated blockchain-based network that makes use of ML techniques.

Further, incorporating the previous work of the standard deviation technique into the estimate helped achieve a more accurate estimate for all stakeholders. Interestingly, the MMRE for the 70%, 80%, and 90% confidence values further improves the MMRE before applying the standard deviation technique.

The complete mathematical model for the software effort estimation case is given below, with its explanation.

$$\widehat{Y}(X) = \frac{1}{N}\sum_{i=1}^{N} T_i(X),$$

$$\widehat{Y}i = w^T X_i + b,$$

$$\widehat{Y}(X) = W^T \varnothing(X) + b,$$

$$\widehat{Y}(X) = \sum_{(k=1)}^{K} f_k(X),\ f_k \in F,$$

$$\text{Expected case} = \frac{\text{best case} + (4 \times \text{most likely}) + \text{worst case}}{6},$$

$$\text{SD} = \frac{\text{Worst case} - \text{best case}}{6},$$

$$70\% = \text{Expected case} + (0.52 \times \text{SD}),$$

$$80\% = \text{Expected case} + (0.84 \times \text{SD}),$$

$$90\% = \text{Expected case} + (1.28 \times \text{SD}).$$

The proposed mathematical model uses the four ML techniques, RF, LR, SVR, and XGBoost, for effort prediction, each represented by its respective formula. These models capture different relationships between input features and the target variable, providing a strong basis for estimation. The use of the standard deviation technique further refines the estimates and presents them with more confidence to the customers. This technique uses the best-case, most-likely, and worst-case scenarios to compute an expected value and confidence intervals (70%, 80%, and 90%). This helps in improving the reliability of the predictions and reducing estimation uncertainty.

6. Conclusion and Future Direction

This research work successfully created an estimation model for the prediction of software effort and completion time that uses ML techniques and elaborates the idea of a conceptual implementation of the model over blockchain technology. The novelty of this system is the conceptual idea of blockchain technology with ML techniques for software estimation. A group of software houses that plan to have a system for software estimation can realize this system. The model has shown the prediction of estimation of time and software effort, but when such a proposed system implements over a blockchain, it will evolve with time into a bigger system covering other estimation aspects like cost, tasks, etc. The proposed system will not only handle the effort and time estimation tasks of software project management, but in a few months, it will comprise of more datasets uploaded by the consortium members. End-users for their estimation cases will use this system, the datasets

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

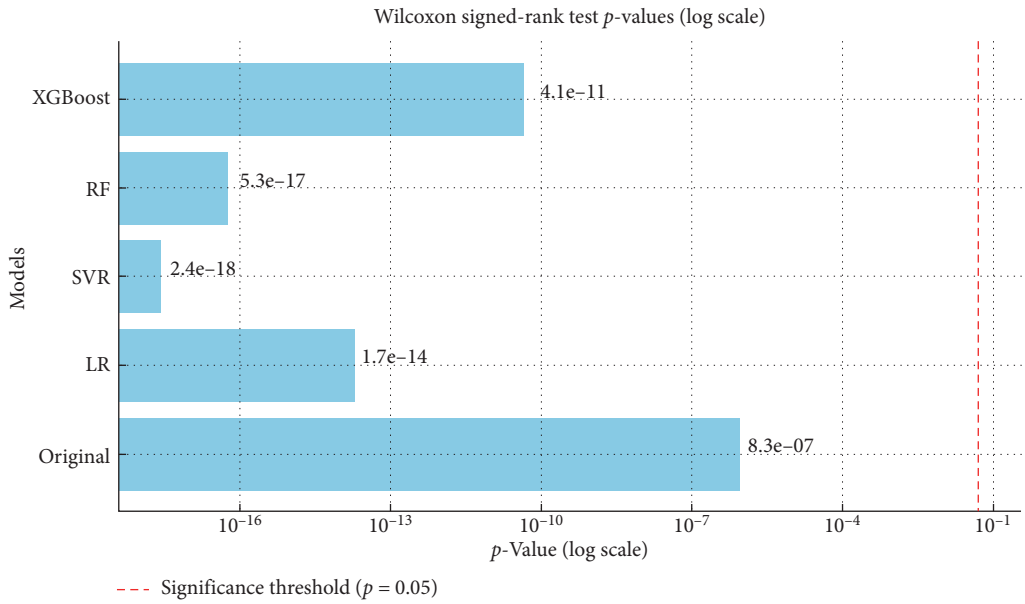

Figure 11: Statistical significance of effort estimation models using the Wilcoxon test (p-values).

Table 9: Standard deviation-based estimation with different confidence levels [1].

Percentage confident (%)	Calculation
70	Expected case + (0.52 × SD)
80	Expected case + (0.84 × SD)
90	Expected case + (1.28 × SD)

will grow with the passage of time, and with information of more projects, the overall estimation for software will improve. The proposed system, when implemented, can help in resource allocation and task scheduling; this can be in the form of the right number of resources and appropriate personnel for the job. This will ultimately help in resource optimization. It can also help in progress tracking and accountability, as the tracking and validation of project milestones are over the blockchain. When there is collaboration among different software houses, the access of shared data between them is secure because of the blockchain technology. The predictions will be refined with the passage of time, and this feedback loop will help in improving the accuracy and reliability of the predictions. At a later point of time, the system will also handle task management, rewarding bonuses to the practitioners, etc. The experimentation results were excellent for the time prediction; the predictions for all four ML techniques improved the MMRE of the original model. The MMRE of the original model is 6.82% while the MMRE of the three ML models is 2.42%, 1.73%, 2.02%, and 3.36% for LR, SVR, RF, and XGBoost, respectively. SVR performs the best with reducing the MMRE to 1.73%, which is phenomenal. The Pred (25) results for all five models are also excellent, with the original model and the three ML models making all 126 projects falling the Pred (25) range. Only in the case of XGBoost 122, projects made the Pred (25) range, which is again very good. For the effort prediction, the original model did not exist, so the four ML techniques recorded MMRE at 25.74%, 20.43%, 24.11%, and 24.19% for LR, SVR, RF, and XGBoost, respectively. From the total of 136 projects for the effort estimation, the Pred (25) results for the four ML models are also not bad, with 83, 88, 77, and 79 projects falling in the Pred (25) range for LR, SVR, RM, and XGBoost, respectively. The LR technique has the least accurate result comparing to other ML models, which is why the use of the standard deviation technique on LR prediction helps in further improving the estimates. The use of the standard deviation techniques helps in further improving the MMRE from 25.74% to 23.1%, 23.8% and 24.5% at 70%, 80%, and 90% confidence levels, respectively. The proposed method for predicting time requires the input of the overall effort of the software for predicting the completion time, which the requesting user provides for the estimation case. Therefore, the main limitation of the proposed method for time estimation is that the prediction time estimate cannot come at an early stage of software development. This means that the requirements should be concretized when the proposed model is used, and the cone of uncertainty narrows to an acceptable range while predicting the software completion time. However, the tested dataset for the prediction of the software effort can predict earlier, as the input features for effort are available quite early in the software development life cycle. A different datasets with the proposed system can also predict the completion time earlier, but the current dataset for time prediction, as it required the software effort as an input feature, could not make a prediction earlier.

The authors intend to work on the proposed model on a real blockchain network and perform the estimations with the actual implementation of smart contracts. The prediction system for project management activities, along with software estimation, can also make the use of the proposed model over a real blockchain network. The other avenues for future

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 10: The best, worst, most likely, and expected cases of linear regression estimation with their standard deviation.

Project	LR_Prediction	Best case	Worst case	Most likely case	Expected value	Standard deviation
1	2.5	2.325	2.75	2.5	2.5125	0.0708
2	5	4.65	5.5	5	5.025	0.1416
3	1	0.93	1.1	1	1.005	0.0283
4	5.75	5.3475	6.325	5.75	5.345417	0.1629
5	2.666667	2.48	2.933333	2.666667	4.302222	0.0755
6	1	0.93	1.1	1	1.005	0.0283
–	–	–	–	–	–	–
–	–	–	–	–	–	–
132	6.3	5.85	6.93	6.3	5.80	0.17
133	2.66	2.48	2.93	2.66	2.83	0.07
134	2.21	2.05	2.43	2.21	2.19	0.062
135	9.5	8.83	10.45	9.5	8.41	0.26
136	1.28	1.19	1.41	1.28	1.16	0.036

Table 11: Linear regression-based estimated times with 70%, 80%, and 90% confidence levels.

Project ID	70% confidence	80% confidence	90% confidence
1	2.5493	2.572	2.6031
2	5.0986	5.144	5.2063
3	1.0197	1.0288	1.0412
4	5.4301	5.482267	5.553
5	4.341	4.365689	4.398
6	1.0197	1.028	1.041
–	–	–	–
132	5.900	5.957	6.035
133	2.874	2.899	2.932
134	2.231	2.251	2.279
135	8.550	8.636	8.754
136	1.187	1.198	1.214

Table 12: MRE and MMRE of the initial estimated effort using linear regression technique, along with the MRE and MMRE of the LR technique at three different confidence levels.

Project ID	MRE LR	MRE LR70	MRE LR80	MRE LR90
1	0.1666	0.150	0.14266	0.1322
2	0.375	0.362	0.357	0.3492
3	0	0.019	0.0288	0.0412
4	0.15	0.08	0.09645	0.11079
5	0.111	0.44	0.45523	0.4663
6	0	0.01	0.0288	0.0412
–	–	–	–	–
132	0.51	0.5461	0.5417	0.5356
133	0.11	0.0417	0.0336	0.02257
134	0.107	0.115	0.1257	0.13957
135	0.9	0.7100	0.7272	0.75094
136	0.28	0.187	0.1989	0.2149
MMRE	25.74%	23.31%	23.8%	24.5%

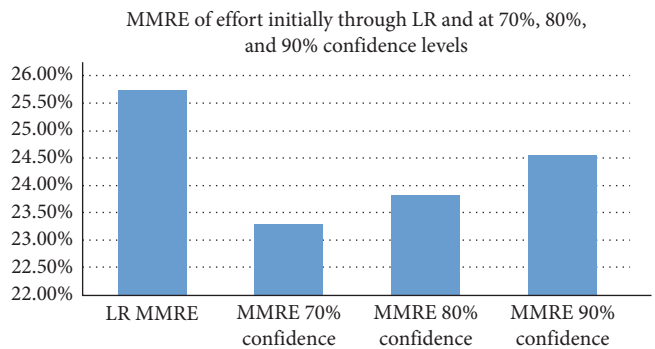

Figure 12: MMRE estimates of effort with different confidence levels of linear regression.

research could be to extend the work to include additional features like task management and resource allocation. Exploring the smart contracts for project milestone validation and incentive distribution. Validating the model on larger and more diverse datasets across industries.

Data Availability Statement

All data are available in the article, and the complete datasets for the experimentations with all the project's data can be found at the following link: https://github.com/ayublatif/Software-Estimation/tree/main.

Conflicts of Interest

The authors declare no conflicts of interest.

Funding

This article was supported by the Education and Research Promotion Program of KOREATECH in 2024.

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

References

[1] S. McConnell, *Software Estimation: Demystifying the Black Art* (Microsoft Press, 2006).

[2] M. A. Latif, M. Y. Khan, and K. Bashir, "Practices for Achieving Accuracy in Software Costing and Estimation," *KIET Journal of Computing and Information Sciences* 1, no. 1 (2018): 13.

[3] C. Northcote Parkinson, *Parkinson's Law and Other Studies in Administration* (Houghton Mifflin, 1957).

[4] C. F. Kemerer, "An Empirical Validation of Software Cost Estimation Models," *Communications of the ACM* 30, no. 5 (1987): 416–429.

[5] B. Boehm, C. Abts, and S. Chulani, "Software Development Cost Estimation Approaches—A Survey," *Annals of Software Engineering* 10, no. 1–4 (2000): 177–205.

[6] T. Menzies, Z. Chen, J. Hihn, and K. Lum, "Selecting Best Practices for Effort Estimation," *IEEE Transactions on Software Engineering* 32, no. 11 (2006): 883–895.

[7] C. H. Rashid, I. Shafi, J. Ahmad, et al., "Software Cost and Effort Estimation: Current Approaches and Future Trends," *IEEE Access* 11 (2023): 99268–99288.

[8] I. F. de Barcelos Tronto, J. D. S. da Silva, and N. Sant'Anna, "An Investigation of Artificial Neural Networks Based Prediction Systems in Software Project Management," *Journal of Systems and Software* 81, no. 3 (2008): 356–367.

[9] N. Bahnas, K. Adel, R. Khallaf, and A. Elhakeem, "Monitoring and Controlling Engineering Projects With Blockchain-Based Critical Chain Project Management," *Automation in Construction* 165 (2024): 105484.

[10] M. A. Akbar, A. Al-Sanad, A. A. AlSanad, A. Ghmaei, M. Shafiq, and T. Kamal, "Towards Efficient and Secure Global Software Development Using Blockchain," in *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*, (Association for Computing Machinery, 2020): 493–498.

[11] M. S. Farooq, M. Ahmed, and M. Emran, "A Survey on Blockchain Acquainted Software Requirements Engineering: Model, Opportunities, Challenges, and Future Directions," *IEEE Access* 10 (2022): 48193–48228.

[12] J. N. Qureshi, M. S. Farooq, U. Ali, and M. Shaheen, "Exploring the Integration of Blockchain and Distributed DevOps for Secure, Transparent, and Traceable Software Development," *IEEE Access* 13 (2023): 15489.

[13] S. Porru, A. Pinna, M. Marchesi, and R. Tonelli, "Blockchain-Oriented Software Engineering: Challenges and New Directions," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, (Buenos Aires, Argentina: IEEE, 2017), 169–171.

[14] M. Beller and J. Hejderup, "Blockchain-Based Software Engineering," in *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, (Montreal, QC, Canada: IEEE, 2019), 53–56.

[15] O. Hazzan, Y. Dubinsky, O. Hazzan, and Y. Dubinsky, "The Agile Manifesto," in *Agile Anywhere: Essays on Agile Projects and Beyond*, (Springer, Cham, 2014): 9–14.

[16] A. Alliance, *Agile Practice Guide, Project Management Institute, 2017: Agile Practice Guide*, 1 (PMI, Bukupedia, 2017).

[17] T. Dybå and T. Dingsøyr, "Empirical Studies of Agile Software Development: A Systematic Review," *Information and Software Technology* 50, no. 9-10 (2008): 833–859.

[18] C. Larman and V. R. Basili, "Iterative and Incremental Developments. A Brief History," *Computer* 36, no. 6 (2003): 47–56.

[19] J. Erickson, K. Lyytinen, and K. Siau, "Agile Modeling, Agile Software Development, and Extreme Programming: The State of Research," *Journal of Database Management* 16, no. 4 (2005): 88–100.

[20] V. Van Hai, H. L. T. K. Nhung, Z. Prokopova, R. Silhavy, and P. Silhavy, "Toward Improving the Efficiency of Software Development Effort Estimation via Clustering Analysis," *IEEE Access* 10 (2022): 83249–83264.

[21] C. Yu and W. Yao, "Robust Linear Regression: A Review and Comparison," *Communications in Statistics-Simulation and Computation* 46, no. 8 (2017): 6261–6282.

[22] S. S. Ali, J. Ren, K. Zhang, J. Wu, and C. Liu, "Heterogeneous Ensemble Model to Optimize Software Effort Estimation Accuracy," *IEEE Access* 11 (2023): 27759–27792.

[23] N. Rankovic, D. Rankovic, M. Ivanovic, and L. Lazic, "A New Approach to Software Effort Estimation Using Different Artificial Neural Network Architectures and Taguchi Orthogonal Arrays," *IEEE Access* 9 (2021): 26926–26936.

[24] H. D. P. De Carvalho, R. Fagundes, and W. Santos, "Extreme Learning Machine Applied to Software Development Effort Estimation," *IEEE Access* 9 (2021): 92676–92687.

[25] A. Sharma and N. Chaudhary, "Prediction of Software Effort by Using Non-Linear Power Regression for Heterogeneous Projects Based on use Case Points and Lines of Code," *Procedia Computer Science* 218 (2023): 1601–1611.

[26] A. B. Nassif, M. Azzeh, A. Idri, and A. Abran, "Software Development Effort Estimation Using Regression Fuzzy Models," *Computational Intelligence and Neuroscience* 2019 (2019): 8367214, 17.

[27] N. Mittas and L. Angelis, "Visual Comparison of Software Cost Estimation Models by Regression Error Characteristic Analysis," *Journal of Systems and Software* 83, no. 4 (2010): 621–637.

[28] H. Phan and A. Jannesari, "Heterogeneous Graph Neural Networks for Software Effort Estimation," in *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, (Association for Computing Machinery, 2022): 103–113.

[29] M. S. Khan, F. Jabeen, S. Ghouzali, Z. Rehman, S. Naz, and W. Abdul, "Metaheuristic Algorithms in Optimizing Deep Neural Network Model for Software Effort Estimation," *IEEE Access* 9 (2021): 60309–60327.

[30] C. S. Yadav and R. Singh, "Prediction Model for Object Oriented Software Development Effort Estimation Using One Hidden Layer Feed Forward Neural Network With Genetic Algorithm," *Advances in Software Engineering* 2014 (2014): 284531, 6.

[31] A. Iftikhar, M. Alam, R. Ahmed, S. Musa, and M. M. Su'ud, "Risk Prediction by Using Artificial Neural Network in Global Software Development," *Computational Intelligence and Neuroscience* 2021 (2021): 2922728, 25.

[32] L. L. Minku and X. Yao, "An analysis of Multi-Objective Evolutionary Algorithms for Training Ensemble Models Based on Different Performance Measures in Software Effort Estimation," in *Proceedings of the 9th International Conference on Predictive Models in Software Engineering*, (Association for Computing Machinery, 2013): 1–10.

[33] M. O. Elish, T. Helmy, and M. I. Hussain, "Empirical Study of Homogeneous and Heterogeneous Ensemble Models for Software Development Effort Estimation," *Mathematical Problems in Engineering* 2013 (2013): 312067, 21.

[34] A. Saeed, W. H. Butt, F. Kazmi, and M. Arif, "Survey of Software Development Effort Estimation Techniques," in *Proceedings of*

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

the 2018 7th International Conference on Software and Computer Applications, (Association for Computing Machinery, 2018): 82–86.

[35] M. Shepperd and C. Schofield, "Estimating Software Project Effort Using Analogies," *IEEE Transactions on Software Engineering* 23, no. 11 (1997): 736–743.

[36] A. Ali and C. Gravino, "A Systematic Literature Review of Software Effort Prediction Using Machine Learning Methods," *Journal of Software: Evolution and Process* 31, no. 10 (2019): e2211.

[37] M. A. Latif, M. K. Khan, and U. Hani, "Using Standard Deviation With Analogy-Based Estimation for Improved Software Effort Prediction," *KSII Transactions on Internet\\& Information Systems* 17, no. 5 (2023).

[38] M. Turic, S. Celar, S. Dragicevic, and L. Vickovic, "Advanced Bayesian Network for Task Effort Estimation in Agile Software Development," *Applied Sciences* 13, no. 16 (2023): 9465.

[39] Meiliana, G. Daniella, N. Wijaya, N. G. E. Putra, and R. Efata, "Agile Software Development Effort Estimation Based on Product Backlog Items," *Procedia Computer Science* 227 (2023): 186–193.

[40] S. Hameed, Y. Elsheikh, and M. Azzeh, "An Optimized Case-Based Software Project Effort Estimation Using Genetic Algorithm," *Information and Software Technology* 153 (2023): 107088.

[41] E. Rodríguez Sánchez, E. F.ón Vázquez Santacruz, and H. Cervantes Maceda, "Effort and Cost Estimation Using Decision Tree Techniques and Story Points in Agile Software Development," *Mathematics* 11, no. 6 (2023): 1477.

[42] M. Iqbal, M. Ijaz, T. Mazhar, et al., "Exploring Issues of Story-Based Effort Estimation in Agile Software Development (ASD)," *Science of Computer Programming* 236 (2024): 103114.

[43] Y.-F. Li, M. Xie, and T. N. Goh, "A Study of Project Selection and Feature Weighting for Analogy Based Software Cost Estimation," *Journal of Systems and Software* 82, no. 2 (2009): 241–252.

[44] M. Shanker, J. Jaya, and K. Thanushkodi, "An Effective Approach to Software Cost Estimation Based on Soft Computing Techniques," *International Arab Journal of Information Technology (IAJIT)* 12 (2015).

[45] P. Phannachitta, J. Keung, A. Monden, and K. Matsumoto, "A Stability Assessment of Solution Adaptation Techniques for Analogy-Based Software Effort Estimation," *Empirical Software Engineering* 22, no. 1 (2017): 474–504.

[46] A. Idri, M. Hosni, and A. Abran, "Improved Estimation of Software Development Effort Using Classical and Fuzzy Analogy Ensembles," *Applied Soft Computing* 49 (2016): 990–1019.

[47] D. Manikavelan and R. Ponnusamy, "Minimizing Analogy Errors With the Help of Fuzzy," *International Journal of Applied Engineering Research* 13, no. 6 (2018): 4527–4530.

[48] J. H. Wu and J. W. Keung, "Utilizing Cluster Quality in Hierarchical Clustering for Analogy-Based Software Effort Estimation," in *2017 8th IEEE International Conference on Software Engineering and Service Science (ICSESS)*, (Beijing, China: IEEE, 2017), 1–4.

[49] V. Nguyen, T. Huynh, B. Boehm, L. Huang, and T. Truong, "Investigating the use of Duration-Based Windows and Estimation By Analogy for COCOMO," *Journal of Software: Evolution and Process* 31, no. 10 (2019): e2176.

[50] P. Phannachitta, "On an Optimal Analogy-Based Software Effort Estimation," *Information and Software Technology* 125 (2020): 106330.

[51] M. Dashti, T. Javdani Gandomani, D. Hasanpoor Adeh, H. Zulzalil, and A. B. Md Sultan, "LEMABE: A Novel Framework to Improve Analogy-Based Software Cost Estimation Using Learnable Evolution Model," *PeerJ Computer Science* 7 (2022): e800.

[52] S. Ezghari and A. Zahi, "Uncertainty Management in Software Effort Estimation Using a Consistent Fuzzy Analogy-Based Method," *Applied Soft Computing* 67 (2018): 540–557.

[53] S. D. N. S. S. B. Rumjaun, K. A. Gutteea, and L. Nagowah, "Effortest-an Enhanced Software Effort Estimation by Analogy Method," *Adbu Journal of Engineering Technology* 5, no. 2 (2016).

[54] F. Fellir, K. Nafil, R. Touahni, and L. Chung, "Improving Case Based Software Effort Estimation Using a Multi-Criteria Decision Technique," in *Software Engineering and Algorithms in Intelligent Systems: Proceedings of 7th Computer Science On-Line Conference 2018*, ed. R. Silhavy, 17, (Springer, Cham, 2019): 438–451.

[55] F. A. Amazal, A. Idri, and A. Abran, "Software Development Effort Estimation Using Classical and Fuzzy Analogy: A Cross-Validation Comparative Study," *International Journal of Computational Intelligence and Applications* 13, no. 3 (2014): 1450013.

[56] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," *Decentralized Business Review*, 2008.

[57] M. Saad, M. Bin Ahmad, and M. Asif, "Blockchain and IIoT Enabled Solution for Social Distancing and Isolation Management to Prevent Pandemics," *Computers, Materials & Continua* 76, no. 1 (2023): 687–709.

[58] M. Asif, Z. Aziz, M. Bin Ahmad, A. Khalid, H. A. Waris, and A. Gilani, "Blockchain-Based Authentication and Trust Management Mechanism for Smart Cities," *Sensors* 22, no. 7 (2022): 2604.

[59] M. Saad, M. B. Ahmad, M. Asif, M. K. Khan, T. Mahmood, and M. T. Mahmood, "Blockchain-Enabled VANET for Smart Solid Waste Management," *IEEE Access* 11 (2023): 5679–5700.

[60] M. Saad, M. K. Khan, and M. B. Ahmad, "Blockchain-Enabled Vehicular Ad Hoc Networks: A Systematic Literature Review," *Sustainability* 14, no. 7 (2022): 3919.

[61] M. Ahmed, N. Iqbal, F. Hussain, et al., "Blockchain-Based Software Effort Estimation: An Empirical Study," *IEEE Access* 10 (2022): 120412–120425.

[62] M. S. Farooq and F. Ahmed, "A Blockchain-Based Framework for Distributed Agile Software Testing Life Cycle," 2023, https://arxiv.org/abs/2307.07212 [Online]. Available:.

[63] M. Assiri and M. Humayun, "A Blockchain-Enabled Framework for Improving the Software Audit Process," *Applied Sciences* 13, no. 6 (2023): 3437.

[64] K. U. Sarker, R. Hasan, A. B. Deraman, and S. Mahmmod, "A Distributed Software Project Management Framework," *Journal of Advances in Information Technology* 14, no. 4 (2023): 685–693.

[65] U. Farooq, M. Ahmed, S. Hussain, F. Hussain, A. Naseem, and K. Aslam, "Blockchain-Based Software Process Improvement (BBSPI): An Approach for SMEs to Perform Process Improvement," *IEEE Access* 9 (2021): 10426–10442.

[66] J. N. Qureshi, M. S. Farooq, and M. Shuaib, "ChainAgile: A Framework for the Improvement of Scrum Agile Distributed Software Development Based on Blockchain," *PLoS ONE* 19, no. 3 (2024): e0299324.

[67] D. Dzhalila, D. Siahaan, R. Fauzan, R. Asyrofi, and M. I. Karimi, "A Systematic Literature Review on Blockchain Technology in Software Engineering," *Jurnal ELTIKOM* 7, no. 1 (2023): 38–49.

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw/2932863 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

[68] I. Erliyani and S. Wulandari, "An Agile Software Engineering Method to Design Blockchain Applications," *Blockchain Frontier Technology* 1, no. 2 (2022): 84–95.
[69] X. Li, P. Jiang, T. Chen, X. Luo, and Q. Wen, "A Survey on the Security of Blockchain Systems," *Future Generation Computer Systems* 107 (2020): 841–853.
[70] D. Siddarth, D. Allen, and G. Weyl, "The Web3 Decentralization Debate Is Focused on the Wrong Question," *Wired*, 2022.
[71] Q. Hu, M. R. Asghar, and S. Zeadally, "Blockchain-Based Public Ecosystem for Auditing Security of Software Applications," *Computing* 103, no. 11 (2021): 2643–2665.
[72] K. Yuan, Y. Yan, L. Shen, Q. Tang, and C. Jia, "Blockchain Security Research Progress and Hotspots," *IAENG International Journal of Computer Science* 49, no. 2 (2022).
[73] S. Suhail, R. Hussain, R. Jurdak, and C. S. Hong, "Trustworthy Digital Twins in the Industrial Internet of Things With Blockchain," *IEEE Internet Computing* 26, no. 3 (2022): 58–67.
[74] Y. Huang, Y. Bian, R. Li, J. L. Zhao, and P. Shi, "Smart Contract Security: A Software Lifecycle Perspective," *IEEE Access* 7 (2019): 150184–150202.
[75] S. K. T. Ziauddin and S. Zia, "An Effort Estimation Model for Agile Software Development," *Advances in Computer Science and Its Applications (ACSA)* 2, no. 1 (2012): 314–324.
[76] H. A. R. M. Al-Ansary, "A Proposed Genetic Algorithm Model to Improve Effort Estimation in Agile Software Development," *International Journal of Computer Science and Information Security (IJCSIS)* 22, no. 4 (2024).
[77] V. Tawosi, R. Moussa, and F. Sarro, "Agile Effort Estimation: Have We Solved the Problem Yet? Insights From a Replication Study," *IEEE Transactions on Software Engineering* 49, no. 4 (2023): 2677–2697.
[78] M. Durán, R. Juárez-Ramírez, S. Jiménez, and C. Tona, "User Story Estimation Based on the Complexity Decomposition Using Bayesian Networks," *Programming and Computer Software* 46, no. 8 (2020): 569–583.
[79] B. Alsaadi and K. Saeedi, "Ensemble Effort Estimation for Novice Agile Teams," *Information and Software Technology* 170 (2024): 107447.

iesfw, 2025, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/sfw2/9238663 by Zekeriya Anil Guven - New Aarhus University , Wiley Online Library on [26/01/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License